

IT AUTOMATION INCIDENT TICKET MANAGER

1. Program Title

IT Automation Incident Ticket Manager

2. Objective

To develop an IT Automation Incident Ticket Manager that applies linear data structure operations to organize and manage incident tickets generated by automation bots.

3. Scenario

The program simulates an IT support team managing incident tickets generated by automation bots. Each ticket contains an Incident ID, Bot, and Short Description. The system allows users to add, display, search, remove, and count active incident tickets.

4. Data Structure Used

The program uses a Python list as its linear data structure. Each incident ticket is stored as a dictionary containing three pieces of information: Incident ID, Bot, and Short Description. The list allows the program to store multiple tickets and perform operations such as adding, searching, displaying, removing, and counting tickets.

5. Program Operations

1. Add New Incident Ticket

Allows the user to enter a new Incident ID, Bot, and Short Description. The new ticket is added to the list of active tickets.

2. Display All Active Incident Tickets

Displays all currently active incident tickets, including their Incident ID, Bot, and Short Description.

3. Search Incident Ticket

Allows the user to search for a specific ticket using its Incident ID. If the ticket exists, its details are displayed.

4. Remove Resolved Incident Ticket

Allows the user to remove a resolved ticket from the list using its Incident ID.

5. Display Total Active Incident Tickets

Displays the total number of tickets currently stored in the list.

6. Exit

Ends the program.

6. Source Code

The complete Python source code used for the IT Automation Incident Ticket Manager is shown below.

```
tickets = []

tickets.append({
    "id": "INC1392939",
    "bot": "BOT-Inventory",
    "description": "Failed to generate the daily report"
})

tickets.append({
    "id": "INC1392940",
    "bot": "BOT-Email",
    "description": "Failed to send the scheduled notification"
})

tickets.append({
    "id": "INC1392941",
    "bot": "BOT-DataSync",
    "description": "Encountered an error during data transfer"
})

tickets.append({
    "id": "INC1392942",
    "bot": "BOT-Invoice",
    "description": "Failed to process an invoice"
})

tickets.append({
    "id": "INC1392943",
    "bot": "BOT-Report",
    "description": "Failed to generate the weekly report"
})

tickets.append({
    "id": "INC1392944",
    "bot": "BOT-FileTransfer",
    "description": "Failed to upload the required file"
})

tickets.append({
    "id": "INC1392945",
    "bot": "BOT-DataEntry",
    "description": "Encountered an error while entering records"
})

tickets.append({
    "id": "INC1392946",
    "bot": "BOT-Backup",
    "description": "Failed to complete the scheduled backup"
})

tickets.append({
    "id": "INC1392947",
    "bot": "BOT-Validation",
    "description": "Failed to validate the submitted records"
})

tickets.append({
    "id": "INC1392948",
    "bot": "BOT-Notification",
    "description": "Failed to send the system alert"
})

def display_tickets():
    print("\n--- Active Incident Tickets ---")

    for ticket in tickets:
        print(f"ID: {ticket['id']}")
        print(f"Bot: {ticket['bot']}")
        print(f>Description: {ticket['description']}")
        print()

def add_ticket():
    incident_id = input("Enter Incident ID: ")
    bot = input("Enter Bot: ")
    description = input("Enter Short Description: ")
```

```

new_ticket = {
    "id": incident_id,
    "bot": bot,
    "description": description
}

tickets.append(new_ticket)

print("Incident ticket added successfully!")

def search_ticket():
    search_id = input("Enter Incident ID to search: ")

    for ticket in tickets:
        if ticket["id"] == search_id:
            print("\nTicket Found!")
            print(f"ID: {ticket['id']}")
            print(f"Bot: {ticket['bot']}")
            print(f>Description: {ticket['description']}")
            return

    print("Ticket not found.")

def remove_ticket():
    remove_id = input("Enter Incident ID to remove: ")

    for ticket in tickets:
        if ticket["id"] == remove_id:
            tickets.remove(ticket)
            print("Incident ticket removed successfully!")
            return

    print("Ticket not found.")

def count_tickets():
    print(f"Total active incident tickets: {len(tickets)}")

while True:
    print("\n=== IT Automation Incident Ticket Manager ===")
    print("1. Add New Incident Ticket")
    print("2. Display All Active Incident Tickets")
    print("3. Search Incident Ticket")
    print("4. Remove Resolved Incident Ticket")
    print("5. Display Total Active Incident Tickets")
    print("6. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_ticket()

    elif choice == "2":
        display_tickets()

    elif choice == "3":
        search_ticket()

    elif choice == "4":
        remove_ticket()

    elif choice == "5":
        count_tickets()

    elif choice == "6":
        print("Thank you for using the Incident Ticket Manager!")
        break

    else:
        print("Invalid choice. Please try again.")

```

7. Program Output

The program was tested using the provided sample incident tickets and additional test operations. The following outputs were successfully demonstrated:

Display All Active Incident Tickets

The 10 sample incident tickets were displayed successfully. Screenshot: 01_display_tickets.png

Search Incident Ticket

Incident ID INC1392942 was successfully found and displayed with BOT-Invoice and its description. Screenshot: 02_search_ticket.png

Add New Incident Ticket

A test ticket (INC1392949, BOT-Test, Test incident ticket) was successfully added. Screenshot: 03_add_ticket.png

Remove Resolved Incident Ticket

The test ticket INC1392949 was successfully removed. Screenshot: 04_remove_ticket.png

Display Total Active Incident Tickets

The program correctly displayed 10 active incident tickets after the add/remove test. Screenshot: 05_count_tickets.png

The actual screenshots are stored in the GitHub repository inside the LAB EXER 1/screenshots folder.

8. Conclusion

The IT Automation Incident Ticket Manager was successfully developed using Python and a linear data structure. The program can add, display, search, remove, and count incident tickets through a menu-driven interface. The required operations were tested using the provided sample tickets, and the program produced the expected results.